

Transformer Models and Self-Attention in Natural Language Processing

Document Summary

This note explores the evolution of language models, from the limitations of traditional approaches to the revolutionary **Transformer architecture**. It details key innovations like **ELMo** for contextual embeddings, the **Encoder-Decoder** framework, and the pivotal **Attention mechanism**. The document culminates in an in-depth look at the **Transformer**, its **Self-Attention** and **Multi-Head Attention** components, **Positional Encoding**, and its profound impact on modern NLP through models like GPT and BERT.

Transformer Models and Self-Attention in Natural Language Processing

Limitations of Traditional Models

Traditional language models face significant challenges, particularly in:

- Capturing the nuanced **semantic meaning** of words and phrases.
- Effectively handling **compositionality**, which refers to how the meaning of a whole is derived from the meaning of its parts, and **long-term dependencies**, where the meaning of a word depends on words far away in the sequence.
- Accurately modeling **polysemy**, which is the phenomenon where a single word has multiple related meanings.

Practical Example

Esempio di Polysemy: Consider the word "mouse," "cell," or "dividend" in different contexts:

- **mouse** : "The **mouse** is not working." (referring to a computer input device) vs. "The cat ate the **mouse**." (referring to an animal).
- **cell** : "The **cells** of a honeycomb." (a hexagonal compartment) vs. "Mobile **cell**." (a mobile phone) vs. "Prison **cell**." (a small room for an inmate).
- **dividend** : "A **dividend** is a distribution of cash..." (a financial payment) vs. "In **division**, the amount... is the **dividend**." (a number to be divided).

Bidirectional Language Models and ELMo

ELMo (Embeddings from Language Models) is a significant advancement in language modeling.

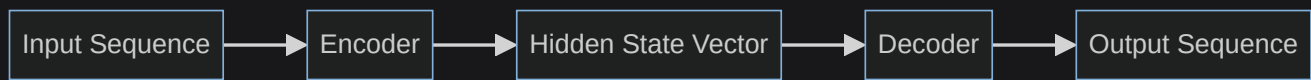
- **Reference:** Introduced by Peters et al. in 2017.
- **Concept:** ELMo models are pretrained on a large text corpus. They are designed to capture context from both previous and next words in a sentence by using **deep bidirectional language models**. The different layers within ELMo capture various linguistic aspects, from lower-level **syntax** to higher-level **semantics**.
 - **Forward Language Model (LM):** Predicts the next word based on preceding words. This is represented as $P(t_k | t_1, \dots, t_{k-1})$.
 - **Backward Language Model (LM):** Predicts the previous word based on succeeding words. This is represented as $P(t_k | t_{k+1}, \dots, t_N)$.
- ELMo uses shared parameters to generate token representations, making the model more efficient.
- Each hidden layer in ELMo is a **bidirectional LSTM** (Long Short-Term Memory) network, allowing it to process information in both directions.
- **Application:** For specific downstream tasks, **softmax** weights are applied in a task-dependent manner to combine the representations from different ELMo layers into a single vector. This process maximizes the log likelihood of both the forward and backward language models, ensuring a rich contextual representation.
- **Tasks:** ELMo has been successfully applied to various NLP tasks, including question answering, sentiment analysis, named entity extraction, semantic role labeling, and conflict resolution.
- **Details:** For a deeper dive into the ELMo architecture and its performance, refer to the original paper: <https://arxiv.org/pdf/1802.05365.pdf>

Encoder-Decoder Architecture

The **Encoder-Decoder architecture** is a fundamental framework in sequence-to-sequence tasks.

- **Encoder:** The encoder's role is to process the input sequence and condense its context into a fixed-size **hidden state vector**. This vector then serves as the input or starting point for the decoder.
 - Esempio: If the input sentence is "The human control the machine", the encoder might produce a vector like `[0.45, 20.4, 6.92, 2.87]`. This vector encapsulates the meaning of the entire sentence.
- **Sentence encoders** are specifically designed to produce representations that capture the meaning of whole sentences, rather than individual words.

- **Decoder:** The decoder then takes this hidden state vector from the encoder and generates the output sequence, typically processing it word by word, from left-to-right (or sometimes right-to-left).



Applications: This architecture is widely used in tasks such as **machine translation**, **question answering**, **summarization**, and **text paraphrasing**.

Attention Mechanisms

The **Attention mechanism** is a crucial innovation that allows models to focus on specific parts of the input sequence when generating each part of the output sequence.

- **Core Idea:** Instead of relying solely on a fixed-size hidden state, attention mechanisms dynamically weigh the importance of different input elements.
- **Similarity Score (S_i):** This score measures the relevance between a **query** (q) and each **key** (k_i) from the input sequence. Several methods exist for computing this score:
 - **Additive Attention:** $S_i = w_3 \tanh(w_2^T q + w_1^T k_i)$, where w_1, w_2, w_3 are learned weight matrices.
 - **Dot Product Attention:** $S_i = q^T k_i$. This is a simpler and often effective method.
 - **Scaled Dot Product Attention:** $S_i = q^T k_i / d_k^{1/2}$, where d_k is the dimension of the key vectors. Scaling by $d_k^{1/2}$ helps to prevent the dot products from becoming too large, especially with high-dimensional vectors, which can push the softmax function into regions with very small gradients.
- **Probabilities (P_i):** Once similarity scores are computed, they are converted into probabilities using the **softmax** function: $P_i = \text{softmax}(S_i)$. These probabilities represent the attention weights, indicating how much focus should be given to each input element.

Practical Example

Esempio di Calcolo dei Pesi di Attenzione: Immaginiamo di avere un query $q = [1, 0]$ e due keys $k_1 = [0.5, 0.5]$ e $k_2 = [1, -1]$. Usando il dot product attention:

$$S_1 = q^T k_1 = (1 \times 0.5) + (0 \times 0.5) = 0.5$$

$$S_2 = q^T k_2 = (1 \times 1) + (0 \times -1) = 1.0$$

Ora, applichiamo la funzione softmax per ottenere le probabilità P_i :

$$P_1 = \frac{e^{S_1}}{e^{S_1} + e^{S_2}} = \frac{e^{0.5}}{e^{0.5} + e^{1.0}} \approx \frac{1.648}{1.648 + 2.718} \approx \frac{1.648}{4.366} \approx 0.377$$

$$P_2 = \frac{e^{S_2}}{e^{S_1} + e^{S_2}} = \frac{e^{1.0}}{e^{0.5} + e^{1.0}} \approx \frac{2.718}{1.648 + 2.718} \approx \frac{2.718}{4.366} \approx 0.623$$

Quindi, il modello presterà più attenzione al secondo key ($P_2 \approx 0.623$) rispetto al primo key ($P_1 \approx 0.377$).

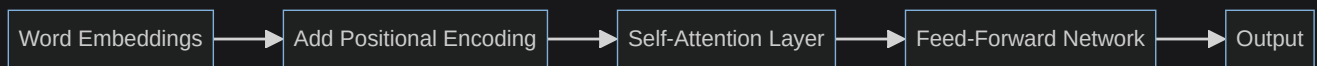
The Transformer Architecture

The **Transformer architecture**, introduced by Vaswani et al. in NIPS 2017, revolutionized sequence modeling.

- **Reference:** "Attention Is All You Need" (Vaswani et al., NIPS 2017).
- **Motivation:** The primary motivation behind the Transformer was to overcome the inherent limitations of Recurrent Neural Networks (RNNs), particularly their sequential processing nature, which hinders parallelization and efficiency.
- **Key Features:**
 - **No Recurrence:** Unlike RNNs, the Transformer completely abandons recurrence. It processes input sequences without using recurrent connections.
 - **Parallel Processing:** It can process entire word sequences in parallel. This parallelization capability makes it significantly more efficient, especially when leveraging modern hardware like GPUs.
 - **Positional Encoding:** Since the Transformer lacks recurrence and convolutions, it needs a mechanism to account for the order of words in a sequence. **Positional Encoding** adds information about the absolute or relative position of tokens. This prevents the model from treating the input as a Bag-Of-Words (BOW-like representation) where word order is lost.
 - The positional encoding for a given position `pos` and dimension `i` is calculated using sine and cosine functions:
 - For even dimensions: $PE_{pos,2i} = \sin(pos/10000^{2i/d_{\text{model}}})$
 - For odd dimensions: $PE_{pos,2i+1} = \cos(pos/10000^{2i/d_{\text{model}}})$
 - An important property is that PE_{pos+k} can be represented as a linear function of PE_{pos} , allowing the model to easily learn relative positions.
 - These positional encodings are simply summed with the word embeddings before being fed into the Transformer layers.
 - **Self-Attention Layer:** This is the core component that allows the Transformer to connect any two positions in the input sequence with a constant number of operations, unlike RNNs which require $O(n)$ operations (where n is the sequence length). The computational complexity of self-attention is quadratic with sequence length n ($O(n^2 \cdot d_{\text{model}})$) but linear with a fixed representation dimension ($O(n \cdot d_{\text{model}})$ if $n < d_{\text{model}}$).
 - **Mechanism:** The input to the self-attention layer is a vector of word subvectors (embeddings combined with positional encodings). For each word, the self-attention mechanism computes attention scores between that word (acting as a **query**) and all other words in the sequence (acting as **keys**). These scores are then used to take a

convex combination of the **values** (which are typically the same as the keys).

Essentially, a **dot product** operation merges information from pairs of words, allowing the model to weigh their relevance to each other.

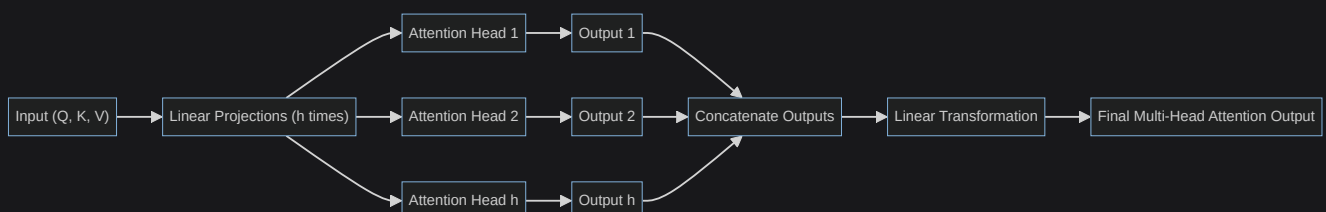


- **Types of Attention:**

- **Self-attention:** This mechanism allows the model to relate different positions of a single sequence to compute a representation for that sequence. In the decoder, self-attention combines output words with previous output words, but it **masks future words** to prevent the model from "cheating" by looking at the answer.
- **Cross-attention:** Used in the decoder, cross-attention combines output words (query) with input words (keys and values) from the encoder's output. This allows the decoder to focus on relevant parts of the source sequence when generating the target sequence.
- **Layer Normalization:** This technique normalizes the values within each layer to have a zero mean and unit variance. It is applied per layer, helping to reduce **gradient dependencies**, speed up **convergence** during training, and avoid excessively large weight adjustments.

Multi-Head Attention

- **Concept: Multi-Head Attention** extends the idea of single attention by allowing the model to jointly attend to information from different representation subspaces at different positions. This is analogous to how Convolutional Neural Networks (CNNs) use multiple **feature maps** to learn diverse patterns.
- **Mechanism:** Instead of performing a single attention function, Multi-Head Attention linearly projects the queries, keys, and values **h** times with different, learned linear projections. For each of these **h** projections, an attention function is applied in parallel, resulting in **h** different "attention heads." The outputs from these **h** heads are then concatenated and linearly transformed again to produce the final output.



- **Masking:** A crucial aspect of multi-head attention in the decoder is **masking**. In the decoder's self-attention sub-layer, future word links are removed (masked out). This

ensures that the prediction for a given word can only depend on the preceding words in the output sequence, maintaining the auto-regressive property.

Impact and Successors

Transformers have had a monumental impact on the field of [Natural Language Processing](#), offering 2-3 orders of magnitude better performance in many tasks compared to previous models. This success has led to the development of a new generation of powerful language models, including:

- **GPT-* ** (Generative Pre-trained Transformer) series by OpenAI.**
- **BERT** (Bidirectional Encoder Representations from Transformers) by Google.
- **DistilBERT**, a distilled version of BERT.
- **ROBERTa**, an optimized version of BERT.
- **XLNet**, which combines the advantages of auto-regressive and auto-encoding models.
- And many other advanced models that continue to push the boundaries of NLP.

References

- [Attention Is All You Need \(Vaswani et al. 2017\)](#)
- [ELMo \(Embeddings from Language Models\)](#)
- [Recurrent Neural Networks](#)
- [Long Short-Term Memory \(LSTM\)](#)
- [Natural Language Processing](#)
- [Machine Learning](#)